ADVANCED WEB DEVELOPER ASSESSMENT • EXPERIENCE LEVEL: 2–4 YEARS

PinkSync — Project & Team Management System

Complete Engineering Architecture, Code Walkthrough, and Technical Interview Preparation Guide

Candidate: A. Madhulatha (8106039645) • **Email:** annavarapumadhulatha4@gmail.com

Live Demo: https://exam-kappa-silk.vercel.app/login • **GitHub:** https://github.com/Madhu81060/exam-

1 Project Overview & Technology Stack

PinkSync is a full-stack, enterprise-grade Project and Team Management System developed using a modern Pink & White aesthetic. The system allows organizations to manage project lifecycles, assign responsibilities to team members, track deliverable progress, and monitor approaching deadlines with strictly enforced role-based permissions.

Technology Stack Summary & Architecture

Frontend Layer	React 18, TypeScript, Vite, Tailwind CSS (Pink & White theme), Lucide Icons.
Backend Layer	Node.js, Express, TypeScript, RESTful decoupled MVC architecture.
Database & ORM	Prisma ORM with SQLite (100% reproducible locally, PostgreSQL compatible).
Security & Auth	JWT (7-day stateless expiry), Bcrypt (10 salt rounds), Zod validation, Rate limiting.
Testing Suite	Vitest + Supertest with 21 unit & integration tests (100% passing).
Cloud Deployment	Frontend on Vercel (SPA rewrites), Backend API on Render (Node 20 LTS).

2 Folder Structure & File Organization

The project employs a clean monorepo structure with clear separation between backend and frontend codebases:

- **Root /:** package.json (orchestration), README.md (Mermaid ER diagram & docs), .gitignore, .node-version.
- **server/src/app.ts:** Express application initialization, CORS configuration, JSON body parsing, and route mounting.
- **server/src/server.ts:** Server startup binding to port 5000 and 0.0.0.0 for cloud hosting compatibility.
- **server/src/controllers/:** Business logic handlers: auth.controller.ts, project.controller.ts, task.controller.ts, dashboard.controller.ts, and user.controller.ts.
- **server/src/middleware/:** auth.ts (JWT verification), rbac.ts (Role-based guards), validate.ts (Zod schemas), rateLimiter.ts (Brute-force protection), and errorHandler.ts (Centralized errors).
- **server/prisma/schema.prisma:** Normalized database schema with 6 entities: User, Project, ProjectMember, Task, TaskComment, and ActivityLog.
- **server/prisma/seed.ts:** Database seeder populating realistic demo projects, tasks, comments, and role credentials.
- **server/tests/:** Vitest integration test suites: auth.test.ts (8 tests), projects.test.ts (6 tests), tasks.test.ts (7 tests).
- **client/src/api/:** Axios API client with interceptors for Bearer JWT token handling and modular endpoint calls.
- **client/src/context/:** AuthContext.tsx providing global authentication state, session restoration, and role helpers.
- **client/src/components/:** Reusable UI components: StatusBadge, PriorityBadge, RoleBadge, Modal, Spinner, Navbar, Sidebar, and interactive drag-and-drop KanbanBoard.
- **client/src/pages/:** Dashboard, Projects, ProjectDetail, Tasks, TaskDetail, TeamManagement, and Login/Register.

3 Role-Based Access Control (RBAC) Permissions

Permissions are strictly enforced on the backend at database query execution time, ensuring unauthorized requests receive HTTP 403 Forbidden even if buttons are tampered with in client browsers:

Role Permissions Matrix

Administrator (ADMIN)	Global control. Can manage all users, reassign roles, create/delete any project or task, and view global organization analytics.
Project Manager (PM)	Can create projects, assign/remove project members, create tasks, and manage workflows for authorized projects. Forbidden from unassigned projects.
Team Member (MEMBER)	Can view assigned projects, update task statuses (TODO -> DONE), and post comments on assigned work. Strictly forbidden from creating projects or deleting tasks.

4 Technical Interview & Code Walkthrough (Section 14)

Detailed, professional answers to the 8 core technical questions specified in the assessment guidelines:

Q1: Explain your application architecture and why you selected it.

I selected a decoupled client-server architecture with React 18 and TypeScript on the frontend and Node.js with Express and Prisma ORM on the backend. This gives clean separation of concerns: routes declare HTTP verbs and endpoints; controllers handle request parameters and responses; middleware enforces authentication, RBAC, and input validation; and Prisma handles type-safe persistence. Prisma with SQLite allows 100% zero-configuration local reproducibility without requiring reviewers to install external PostgreSQL servers, while the schema allows 1-line switching to PostgreSQL.

Q2: Explain your database schema and relationship design.

The schema is normalized into 6 relational entities: Users, Projects, ProjectMembers, Tasks, TaskComments, and ActivityLogs. Foreign keys enforce referential integrity: deleting a project cascades to its tasks and member records; deleting a user sets task assignedToId to NULL via onDelete: SetNull. A unique composite index on ProjectMember [projectId, userId] prevents duplicate memberships, while indexes on status, priority, and due dates optimize query execution.

Q3: How do authentication and role-based authorization work?

Authentication uses salted bcrypt password hashing (10 rounds) and stateless JWT tokens with 7-day expiration. When a client requests a protected endpoint, the authenticateToken middleware extracts the Bearer token, verifies its cryptographic signature, and loads the active user context into req.user. Role-based access control (RBAC) then evaluates whether the user possesses the required role (Admin, Project Manager, or Team Member).

Q4: How do you prevent unauthorized access to projects and tasks?

Security is enforced on the backend at query time, not just by hiding UI buttons. Even if an attacker crafts an arbitrary HTTP request via cURL or Postman, the backend canAccessProject and canManageProject helpers verify that the user is either an Admin, the project creator, or a registered ProjectMember. Non-admin task queries are strictly scoped using Prisma WHERE clauses, returning HTTP 403 Forbidden on violations.

Q5: How are API validation and error handling implemented?

Input validation is handled using Zod schemas for all request bodies and query parameters. If validation fails, a custom middleware formats the Zod error array into clean, field-specific error messages with HTTP 400 Bad Request. All controller errors bubble to a centralized Express error middleware that logs the error and returns consistent JSON responses, suppressing sensitive stack traces in production.

Q6: How do you handle pagination and database query performance?

Endpoints use database-backed pagination via Prisma skip and take arguments, alongside a parallel prisma.count() query. This avoids loading entire datasets into memory. To prevent N+1 query bottlenecks, relational queries utilize Prisma include clauses with select projections, executing optimized SQL joins rather than iterative loops.

Q7: What do your automated tests cover and how are they run?

We implemented 21 automated integration tests using Vitest and Supertest across three suites: auth.test.ts (login, register, duplicate email prevention, JWT verification), projects.test.ts (project CRUD, RBAC 403 checks, unassigned project scoping), and tasks.test.ts (task CRUD, status progression, comments, and database dashboard metrics). All tests run via "npm test".

Q8: What trade-offs did you make and what would you improve with more time?

Trade-offs: SQLite was chosen for zero-friction evaluation; with more time, I would set up a multi-stage Docker Compose pipeline with PostgreSQL. Bonus enhancements would include WebSocket real-time updates (Socket.io) for multi-user Kanban sync, S3 file attachments on task comments, and automated email notifications for approaching deadlines.

Q9: How did you prevent brute-force login attacks?

We integrated express-rate-limit configured as authRateLimiter on /api/auth/login and /api/auth/register, capping attempts to 15 requests per 15-minute window per IP address. In test environments (NODE_ENV=test), the limit dynamically expands to prevent test suite interference.

Q10: How is client-side SPA routing handled when deployed on Vercel?

Because React Router handles routing on the client side, directly requesting or refreshing a nested route like /projects or /tasks on static hosts can result in an HTTP 404 error. We created a vercel.json configuration with rewrite rules mapping all wildcard routes /(.*) to /index.html, ensuring smooth client-side hydration.

5 Automated Test Results & Verification Summary

Vitest + Supertest Execution Results (npm test)

auth.test.ts	8 tests passed: User registration, duplicate email rejection (409), login validation, password verification, /me endpoint.
projects.test.ts	6 tests passed: Project creation by PM, Team Member creation rejection (403), required-field validation, project updates, scoping.
tasks.test.ts	7 tests passed: Task creation, assignee status updates (TODO -> IN_PROGRESS), comments, priority filtering, and live dashboard metrics.
Overall Status	3 Test Files Passed | 21 of 21 Tests Passed (100% Success Rate) | Execution Duration: ~2.46s.

6 Demo Credentials & Submission Contacts

Pre-Seeded Demo User Accounts (Password: Password123!)

Administrator	admin@projecthub.com — Full governance across all projects, tasks, user directory, and roles.
Project Manager	pm@projecthub.com — Create projects, manage team assignments, and direct tasks in authorized projects.
Team Member	member@projecthub.com — View assigned projects, update task statuses, and post task comments.

Submission & Contact Details

Candidate Name	A. Madhulatha
Mobile Number	+91 8106039645
Email Address	annavarapumadhulatha4@gmail.com
Live Application	https://exam-kappa-silk.vercel.app/login
GitHub Repository	https://github.com/Madhu81060/exam-
Backend REST API	https://exam-backend-mxd7.onrender.com/api

PinkSync Assessment Submission • All requirements 100% completed and verified.